

GeCode CSP Pipeline - Componentes del Pipeline

Detalle de Cada Componente del Sistema

Proyecto GeCode CSP Pipeline

2026

Contents

Componentes del Pipeline	4
Flujo General	4
1. SyntaxChecker	5
Proposito	5
Entrada/Salida	5
Reglas de Validacion	5
Codigos de Salida	5
Uso	5
Ejemplo	6
Tecnologias	6
Posicion en el Pipeline	6
2. JsonToGraph	7
Proposito	7
Entrada/Salida	7
Algoritmo Principal	7
Tipos de Nodo AST	7
Nodos de Valor	7
Nodos Relacionales	7
Nodos Logicos	7
Nodos Aritmeticos	8
Nodos Especiales	8
Uso	8
Ejemplo de Transformacion	8
Tecnologias	9
Posicion en el Pipeline	9
3. FunctionChecker	10
Proposito	10
Entrada/Salida	10
Algoritmo de Busqueda	10
Directorios de Busqueda	10
Codigos de Salida	10

Uso	11
Notas	11
Tecnologias	11
Posicion en el Pipeline	11
4. FwdConsistency	12
Proposito	12
Entrada/Salida	12
Algoritmo AC-3	12
Ejemplo de Paso	12
Uso	13
Tecnologias	13
Posicion en el Pipeline	13
5. BwdConsistency	14
Proposito	14
Entrada/Salida	14
Algoritmo HC4-Revise	14
Diferencia con FwdConsistency	14
Ejemplo	14
Uso	15
Tecnologias	15
Posicion en el Pipeline	15
6. CSPEval	16
Proposito	16
Entrada/Salida	16
Tipos de Variable Soportados	16
Algoritmo de Propagacion	16
Ejemplo de Salida	16
Diferencia con FwdConsistency/ForwardChain	17
Uso	17
Tecnologias	17
Posicion en el Pipeline	17
7. ForwardChain	18
Proposito	18
Entrada/Salida	18
Algoritmo de Encadenamiento	18
Estados de Retorno	18
Ejemplo de Steps	18
Diferencia con FwdConsistency	19
Uso	19
Tecnologias	19
Posicion en el Pipeline	19
8. JsonSink	20
Proposito	20
Entrada/Salida	20

Esquema SQLite	20
Funcionalidad	20
Uso	20
Notas	21
Tecnologias	21
Posicion en el Pipeline	21
9. JsonSource	22
Proposito	22
Entrada/Salida	22
Modos de Recuperacion	22
Casos de Uso Tipicos	22
1. Guardar y Reutilizar Grafo	22
2. Resolver CSP desde Grafo Guardado	22
3. Recuperar Resultado Anterior	22
Notas	22
Tecnologias	23
Posicion en el Pipeline	23
Resumen de Componentes	24
Tabla Comparativa	24
Pipelines Tipicos	24
Pipeline Completo con Propagacion	24
Pipeline con Persistencia	24
Pipeline de Evaluacion Rapida	24

Componentes del Pipeline

El pipeline de GeCode CSP esta compuesto por **9 componentes principales** que transforman y procesan problemas CSP desde la entrada JSON hasta la resolucion completa. Cada componente tiene una responsabilidad especifica y bien definida.

Flujo General

JSON Input

- ?
- [1] SyntaxChecker -> Validacion de sintaxis y semantica
- ?
- [2] JsonToGraph -> Construccion de AST y grafo
- ?
- [3] FunctionChecker -> Verificacion de funciones
- ?
- [4] FwdConsistency -> Propagacion forward (AC-3)
- ?
- [5] BwdConsistency -> Propagacion backward (HC4)
- ?
- [6] CSPEval -> Evaluacion rapida
- ?
- [7] ForwardChain -> Encadenamiento iterativo
- ?
- [8] TestGecodeBridge -> Resolucion con Gecode
- ?

Solutions Output

Persistencia:

- [9] JsonSink/JsonSource -> SQLite storage

1. SyntaxChecker

Proposito

Validar la **sintaxis y coherencia semantica** de un archivo JSON de sistema CSP antes de procesarlo con el pipeline. Actua como el primer filtro de calidad para detectar errores tempranos.

Entrada/Salida

Entrada: Archivo JSON con campos **variables, expressions, functions**.

Salida: JSON con resultado de validacion:

```
{
  "status": "ok"
}
```

O en caso de errores:

```
{
  "status": "error",
  "errors": [
    {
      "rule": "nombre_regla",
      "msg": "descripcion del error"
    }
  ]
}
```

Reglas de Validacion

El SyntaxChecker verifica las siguientes reglas:

1. **Campos obligatorios presentes:** **variables, expressions**
2. **Variables referenciadas existen:** Todas las variables usadas en constraints estan declaradas
3. **Funciones usadas estan declaradas:** Referencias a funciones resuelven correctamente
4. **Referencias validas:** Variables y funciones se resuelven sin ambigüedades
5. **Sintaxis parseable:** Cada constraint tiene sintaxis valida
6. **Coherencia de dominios:** El campo **value** es subconjunto del **domain**
7. **Aridad correcta:** Funciones user-defined tienen el numero correcto de parametros
8. **No sobrescritura:** No se sobrescriben built-ins del motor (**abs, dist, etc.**)

Codigos de Salida

- 0 = validacion exitosa
- 1 = errores de validacion encontrados

Uso

```
# Validar archivo
./bin/SyntaxChecker sistema.json
```

En el pipeline

```
./bin/SyntaxChecker sistema.json && ./bin/JsonToGraph sistema.json | ...
```

Ejemplo

```
$ ./bin/SyntaxChecker ejemplos/Json_input_1.json
{
  "status": "ok"
}

$ echo $?
0
```

Tecnologias

- **Pascal + MiniJSON** para parsing JSON
- Validacion semantica custom

Posicion en el Pipeline

ETAPA 1 – Siempre primera antes de JsonToGraph.

2. JsonToGraph

Proposito

Convierte un JSON de sistema CSP en un **grafo de restricciones con AST**. Es la etapa central que transforma expresiones de texto en una estructura de datos procesable por el resto del pipeline.

Entrada/Salida

Entrada: Archivo JSON con **variables**, **expressions**, **functions**.

Salida: JSON con cuatro secciones: - **variables** – variables con id asignado, tipo y dominio - **functions** – funciones user-defined declaradas - **constraints** – cada constraint con su AST (nodos tipados) y referencias a variables/funciones usadas - **adjacency** – indice inverso: que constraints afectan a cada variable

Algoritmo Principal

Utiliza un **Pratt Parser** para construir el arbol de sintaxis abstracta de cada expresion:

1. **Tokenizacion:** Divide la expresion en tokens
2. **Parsing por precedencia:** Construye AST respetando precedencia de operadores
3. **Construccion de grafo:** Genera nodos y aristas del grafo de dependencias
4. **Indexacion:** Crea indice inverso de que constraints afectan a cada variable

Tipos de Nodo AST

El parser genera los siguientes tipos de nodos:

Nodos de Valor

- **Variable** – referencia a variable
- **Number** – literal numerico
- **Set** – conjunto literal
- **Interval** – intervalo [a, b]

Nodos Relacionales

- **Equals** – operador =
- **NotEquals** – operador <>
- **Less** – operador <
- **Greater** – operador >
- **LessEq** – operador <=
- **GreaterEq** – operador >=

Nodos Logicos

- **And** – operador AND
- **Or** – operador OR
- **Not** – operador NOT

Nodos Aritmeticos

- Add – operador +
- Subtract – operador -
- Multiply – operador *
- Divide – operador /
- Negate – negacion unaria

Nodos Especiales

- FunctionCall – llamada a funcion
- In – operador de pertenencia IN

Uso

```
# Salida a stdout
./bin/JsonToGraph sistema.json

# Salida a archivo
./bin/JsonToGraph sistema.json salida.json

# En el pipeline
./bin/JsonToGraph sistema.json | ./bin/FwdConsistency
```

Ejemplo de Transformacion

Entrada:

```
{
  "variables": [
    {"nombre": "x", "tipo": "integer", "domain": [1, 10]}
  ],
  "expresiones": ["x + y * 2"]
}
```

Salida (AST):

```
{
  "ast": {
    "tipo": "BINOP",
    "op": "+",
    "izq": {"tipo": "IDENT", "nombre": "x"},
    "der": {
      "tipo": "BINOP",
      "op": "*",
      "izq": {"tipo": "IDENT", "nombre": "y"},
      "der": {"tipo": "LITERAL", "valor": 2}
    }
  }
}
```


Tecnologias

- **Pascal + PrattParser + ExpressionAST**
- Parsing por precedencia de operadores
- Construccion de grafo dirigido aciclico

Posicion en el Pipeline

ETAPA 2 – Despues de SyntaxChecker, antes de todo lo demas.

3. FunctionChecker

Proposito

Verifica que los **archivos objeto** (.so / .o) de las funciones user-defined referenciadas en el grafo existan en los directorios de busqueda antes de intentar ejecutar el pipeline.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con resultado:

```
{
  "status": "ok"|"missing"|"error",
  "checked": N,
  "found": N,
  "missing": N,
  "functions": [
    {
      "name": "miFuncion",
      "object": "func_miFuncion.so",
      "path": "./lib/func_miFuncion.so",
      "found": true
    }
  ]
}
```

Algoritmo de Busqueda

1. **Extraer funciones:** Lee la seccion `functions` del grafo JSON
2. **Buscar objetos:** Para cada funcion, busca `func_<nombre>.so` o `func_<nombre>.o` en:
 - `.` (directorio actual)
 - `./lib`
 - `./obj`
 - `/usr/local/lib/csp`
 - Directorios adicionales con `--path`
3. **Reportar resultados:** Marca cada funcion como encontrada o faltante

Directorios de Busqueda

Por defecto: `../lib:./obj:/usr/local/lib/csp`

Puede personalizarse:

```
./bin/FunctionChecker grafo.json --path ./lib:./obj:/usr/local/lib/csp
```

Codigos de Salida

- 0 = todos los objetos encontrados
- 1 = algun objeto faltante (puede continuar con advertencia)

- 2 = error fatal (no se puede continuar)

Uso

```
# Verificacion basica
./bin/FunctionChecker grafo.json

# Con directorios personalizados
./bin/FunctionChecker grafo.json --path ./custom_libs:./obj

# En el pipeline
./bin/JsonToGraph sistema.json | ./bin/FunctionChecker
```

Notas

- Solo relevante si el sistema usa funciones user-defined
- Si no hay funciones, el resultado es siempre “ok”
- Los objetos no se cargan, solo se verifica su existencia

Tecnologias

- **Pascal + UCSPJson**
- Búsqueda de archivos en filesystem

Posicion en el Pipeline

ETAPA 3 – Despues de JsonToGraph, antes de FwdConsistency.

4. FwdConsistency

Proposito

Aplica **consistencia de arco hacia adelante (AC-3)** al grafo de restricciones. Propaga cada constraint desde las variables conocidas hacia las desconocidas, reduciendo dominios progresivamente.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** - “arc_consistent” | “inconsistent” - **queue_ops** - operaciones de cola AC-3 - **firings** - reglas disparadas - **steps** - array de pasos con cada reduccion de dominio - **variables** - dominios finales de todas las variables

Algoritmo AC-3

Arc Consistency Algorithm 3 (AC-3):

1. Inicializar cola Q con todos los arcos (X, C) donde X es variable, C es constraint que involucra a X
2. Mientras Q no este vacia:
 - a. Extraer arco (X, C) de Q
 - b. Si Revise(X, C) reduce el dominio de X:
 - Si domain(X) esta vacio -> INCONSISTENT
 - Agregar a Q todos los arcos (Y, C') donde Y depende de X
3. Si ningun dominio quedo vacio -> ARC_CONSISTENT

Revise(X, C):

Para cada valor v en domain(X):

 Evaluar constraint C con X=v

 Si C no se puede satisfacer con ningun valor de otras variables:

 Eliminar v de domain(X)

Retornar true si algun valor fue eliminado

Ejemplo de Paso

```
{
  "steps": [
    {
      "var": "X",
      "constraint": "X > 5",
      "before": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
      "after": [6, 7, 8, 9, 10]
    }
  ]
}
```

Uso

```
# Salida a stdout  
./bin/FwdConsistency grafo.json  
  
# Salida a archivo  
./bin/FwdConsistency grafo.json salida.json  
  
# Con persistencia  
./bin/FwdConsistency grafo.json | ./bin/JsonSink runs.db fwd
```

Tecnologias

- **Pascal + MiniMath + AC-3 Algorithm**
- Evaluacion de restricciones con aritmetica de intervalos

Posicion en el Pipeline

ETAPA 4 – Despues de FunctionChecker. Complementa a BwdConsistency: ambos se aplican al mismo grafo.

5. BwdConsistency

Proposito

Aplica **consistencia de arco hacia atras (HC4-revise)** al grafo de restricciones. Para constraints del tipo $V = f(W)$, invierte f para restringir W en funcion del dominio reducido de V . Complementa a FwdConsistency.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con la misma estructura que FwdConsistency: - **status** - “arc_consistent” | “inconsistent” - **queue_ops** - operaciones de cola - **firings** - reglas disparadas - **steps** - reducciones de dominio (marcadas con “(bwd)”) - **variables** - dominios finales

Algoritmo HC4-Revise

Hull Consistency 4 (HC4) con proyeccion inversa:

Para cada constraint $V = f(X, Y, Z)$:

1. Forward: Calcular $\text{domain}(V) = f(\text{domain}(X), \text{domain}(Y), \text{domain}(Z))$
2. Backward: Para cada operando (ej. X):
 - a. Proyectar constraint: $X = f_{\text{inv}}(V, Y, Z)$
 - b. Calcular $\text{domain_nuevo}(X) = f_{\text{inv}}(\text{domain}(V), \text{domain}(Y), \text{domain}(Z))$
 - c. Intersectar: $\text{domain}(X) \leftarrow \text{domain}(X) \cap \text{domain_nuevo}(X)$
3. Iterar hasta punto fijo

Diferencia con FwdConsistency

Aspecto	FwdConsistency	BwdConsistency
Direccion	Izquierda -> Derecha	Derecha -> Izquierda
Algoritmo	AC-3 (Arc Consistency)	HC4 (Hull Consistency)
Estrategia	Evaluar constraint con valores	Proyectar funcion inversa
Ejemplo	$x + y = 10 \rightarrow$ reducir y dado x	$z = x + y \rightarrow$ reducir x, y dado z

Juntos cubren **mas reducciones de dominio** que cada uno por separado.

Ejemplo

Constraint: $z = x + y$

Forward:

$x \in [1, 5], y \in [2, 8]$
 $\rightarrow z \in [1+2, 5+8] = [3, 13]$

Backward:

$z \in [5, 10]$ (dominio reducido por otra restriccion)
 $\rightarrow x \in [5-8, 10-2] \cap [1, 5] = [-3, 8] \cap [1, 5] = [1, 5]$ (sin cambio)

-> $y \ ? \ [5-5, 10-1] \ ? \ [2, 8] = [0, 9] \ ? \ [2, 8] = [2, 8]$ (sin cambio)

Uso

```
# Salida a stdout
./bin/BwdConsistency grafo.json

# En paralelo con FwdConsistency
./bin/FwdConsistency grafo.json | ./bin/BwdConsistency

# Con persistencia
./bin/BwdConsistency grafo.json | ./bin/JsonSink runs.db bwd
```

Tecnologías

- **Pascal + MiniMath + Interval Analysis**
- Aritmetica de intervalos con proyeccion inversa

Posicion en el Pipeline

ETAPA 5 – En paralelo o secuencia con FwdConsistency.

6. CSPEval

Proposito

Evaluador CSP generico con propagacion iterativa de dominios. Maneja los cuatro tipos de variable del motor (numeric, integer, boolean, set) con operaciones de reduccion apropiadas para cada tipo.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** – “ok” | “solved” | “contradiction” - **iterations** – iteraciones hasta punto fijo (max 200) - **variables** – dominios finales con flags

Tipos de Variable Soportados

Tipo	Dominio	Operaciones
numeric	Intervalo [lo, hi]	Aritmetica de punto flotante
integer	Enumerado de enteros	Operaciones discretas
boolean	{false, true}	Logica booleana
set	Enumerado de etiquetas	Operaciones de conjuntos

Algoritmo de Propagacion

1. Inicializar dominios de todas las variables
2. Repetir hasta punto fijo (max 200 iteraciones):
 - a. Para cada constraint:
 - Evaluar con dominios actuales
 - Calcular nuevo dominio resultante
 - Intersectar con dominio actual
 - b. Si algun dominio quedo vacio -> CONTRADICTION
 - c. Si todos los dominios tienen 1 solo valor -> SOLVED
 - d. Si ningun dominio cambio -> PUNTO FIJO (status: ok)
3. Retornar dominios finales

Ejemplo de Salida

```
{
  "status": "ok",
  "iterations": 5,
  "variables": [
    {
      "name": "X",
      "type": "integer",
      "domain": [6, 7, 8, 9, 10],
      "solved": false,
      "empty": false
    }
  ]
}
```



```

    },
    {
      "name": "Y",
      "type": "integer",
      "domain": [5],
      "solved": true,
      "empty": false
    }
  ]
}

```

Diferencia con FwdConsistency/ForwardChain

Aspecto	CSPEval	FwdConsistency	ForwardChain
Trazado	No genera steps	Genera steps detallados	Genera steps por iteracion
Velocidad	Mas rapido	Medio	Mas lento
Uso	Evaluacion rapida	Debugging/analisis	Debugging/inferencia

CSPEval produce **solo el resultado final** de dominios, sin trazar los pasos intermedios.

Uso

```

# Evaluacion rapida
./bin/CSPEval grafo.json

# Con persistencia
./bin/CSPEval grafo.json | ./bin/JsonSink runs.db cspeval

```

Tecnologias

- Pascal + MiniMath + Multi-domain evaluation
- Propagacion iterativa hasta punto fijo

Posicion en el Pipeline

Etapas de **evaluacion rapida** antes de pasar a TestGecodeBridge.

7. ForwardChain

Proposito

Encadenamiento hacia adelante iterativo: aplica propagacion de constraints en multiples pasadas hasta alcanzar un punto fijo (ningun dominio cambia) o detectar una contradiccion (dominio vacio).

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** – “ok” | “solved” | “contradiction” - **iterations** – numero de pasadas realizadas - **steps** – array de pasos por iteracion - **variables** – dominios finales

Algoritmo de Encadenamiento

```
iteration = 1
REPEAT:
    changed = false
    FOR EACH constraint C:
        FOR EACH variable V in C:
            domain_old = domain(V)
            domain_new = evaluate_constraint(C, V)
            domain(V) = domain_old ? domain_new
            IF domain(V) != domain_old:
                Log step: {iteration, V, domain_old, domain_new}
                changed = true
            IF domain(V) is empty:
                RETURN "contradiction"
    iteration++
UNTIL NOT changed OR iteration > max_iterations

IF all variables have single value:
    RETURN "solved"
ELSE:
    RETURN "ok"
```

Estados de Retorno

- “solved” – todas las variables tienen exactamente un valor
- “contradiction” – algun dominio quedo vacio (sistema inconsistente)
- “ok” – reduccion parcial alcanzo punto fijo

Ejemplo de Steps

```
{
  "steps": [
    {
      "iteration": 1,
```

```

    "var": "X",
    "before": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "after": [6, 7, 8, 9, 10]
  },
  {
    "iteration": 2,
    "var": "Y",
    "before": [1, 2, 3, 4, 5],
    "after": [1, 2, 3, 4]
  }
]
}

```

Diferencia con FwdConsistency

Aspecto	FwdConsistency	ForwardChain
Estrategia	AC-3 con cola	Multiples pasadas completas
Eficiencia	Mas eficiente	Mas simple
Uso	Produccion	Trazado/debugging
Output	Steps ordenados por cola	Steps ordenados por iteracion

FwdConsistency aplica **AC-3 una sola vez con cola**. ForwardChain hace **multiples pasadas completas** sobre todos los constraints hasta punto fijo.

Uso

```

# Encadenamiento con trazado
./bin/ForwardChain grafo.json

# Con persistencia
./bin/ForwardChain grafo.json | ./bin/JsonSink runs.db forward

```

Tecnologias

- Pascal + MiniMath
- Evaluacion iterativa con punto fijo

Posicion en el Pipeline

Alternativa a FwdConsistency para sistemas con muchas iteraciones o cuando se requiere trazado detallado.

8. JsonSink

Proposito

Sumidero del pipeline: lee JSON de stdin y lo persiste en una base de datos SQLite, devolviendo por stdout un JSON de confirmacion encadenable.

Entrada/Salida

Entrada: JSON producido por cualquier etapa del pipeline (stdin).

Salida: JSON de confirmacion:

```
{
  "ok": true,
  "id": 42,
  "tag": "fwd",
  "status": "arc_consistent"
}
```

Esquema SQLite

```
CREATE TABLE runs (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  ts      TEXT,      -- timestamp ISO-8601 automatico
  tag     TEXT,      -- etiqueta de la etapa
  status  TEXT,      -- campo "status" extraido del JSON
  payload TEXT       -- JSON completo de la etapa
);
```

Funcionalidad

1. **Leer JSON** de stdin
2. **Extraer status** automaticamente del JSON
3. **Insertar registro** en tabla runs con:
 - timestamp automatico
 - tag (argumento 2)
 - status extraido
 - payload completo
4. **Emitir confirmacion** por stdout (encadenable)

Uso

```
# Almacenar JSON
echo '{"data": "value"}' | ./bin/JsonSink db.sqlite mi_tag

# Pipeline con persistencia
./bin/FwdConsistency grafo.json | ./bin/JsonSink pipeline.db fwd
```

```
# Cadena completa
./bin/JsonToGraph input.json | ./bin/JsonSink db.sqlite graph | ./bin/FwdConsistency
```

Notas

- El campo “**status**” se extrae automaticamente del JSON de entrada
- El **payload completo** se almacena para replay o auditoria
- Usa binding de **SQLite3** via cdecl; no requiere runtime adicional
- Para leer los datos persistidos, usar **JsonSource**

Tecnologias

- **Pascal + SQLite3 bindings**
- Insercion transaccional

Posicion en el Pipeline

Cierre de cualquier etapa cuando se desea persistencia.

9. JsonSource

Proposito

Fuente del pipeline: recupera un JSON persistido en SQLite y lo emite por stdout para continuar el pipeline desde un punto guardado.

Entrada/Salida

Entrada: Base de datos SQLite + filtros opcionales.

Salida: El payload JSON original del registro recuperado (stdout).

Modos de Recuperacion

```
# Ultimo registro insertado
./bin/JsonSource runs.db

# Ultimo registro con tag "fwd"
./bin/JsonSource runs.db fwd

# Registro con id=42 y tag="fwd"
./bin/JsonSource runs.db fwd 42
```

Casos de Uso Tipicos

1. Guardar y Reutilizar Grafo

```
# Guardar grafo
./bin/JsonToGraph sistema.json | ./bin/JsonSink pipeline.db graph

# Correr FwdConsistency y BwdConsistency desde el mismo grafo
./bin/JsonSource pipeline.db graph | ./bin/FwdConsistency | ./bin/JsonSink pipeline.db fwd
./bin/JsonSource pipeline.db graph | ./bin/BwdConsistency | ./bin/JsonSink pipeline.db bwd
```

2. Resolver CSP desde Grafo Guardado

```
# Resolver desde grafo persistido
./bin/JsonSource pipeline.db graph | ./bin/TestGecodeBridge
```

3. Recuperar Resultado Anterior

```
# Analizar resultado de etapa anterior
./bin/JsonSource pipeline.db csp 17 | python3 analizar.py
```

Notas

- Si no existe el tag o id, termina con **error en stderr** y **exit 1**
- Usa la misma binding minima de SQLite3 que JsonSink

- Permite **replay** y **debug** de cualquier etapa

Tecnologias

- **Pascal + SQLite3 bindings**
- Query transaccional

Posicion en el Pipeline

Inicio de cualquier etapa cuando el input viene de SQLite. Complemento de JsonSink – juntos implementan **memoria persistente**.

Resumen de Componentes

Tabla Comparativa

Componente	Tipo	Entrada	Salida	Proposito Principal
SyntaxChecker	Validacion	JSON raw	JSON validado	Verificar sintaxis y semantica
JsonToGraph	Transformacion	JSON validado	Grafo + AST	Construir estructura procesable
FunctionChecker	Verificacion	Grafo JSON	Reporte	Verificar objetos de funciones
FwdConsistency	Propagacion	Grafo JSON	Grafo + dominios	AC-3 forward
BwdConsistency	Propagacion	Grafo JSON	Grafo + dominios	HC4 backward
CSPEval	Evaluacion	Grafo JSON	Dominios finales	Evaluacion rapida
ForwardChain	Propagacion	Grafo JSON	Grafo + steps	Encadenamiento iterativo
JsonSink	Persistencia	JSON (stdin)	Confirmacion	Almacenar en SQLite
JsonSource	Persistencia	SQLite + filtros	JSON (stdout)	Recuperar desde SQLite

Pipelines Tipicos

Pipeline Completo con Propagacion

```
./bin/SyntaxChecker input.json && \  
./bin/JsonToGraph input.json | \  
./bin/FunctionChecker | \  
./bin/FwdConsistency | \  
./bin/BwdConsistency | \  
./bin/TestGecodeBridge
```

Pipeline con Persistencia

```
./bin/JsonToGraph input.json | ./bin/JsonSink db.sqlite graph  
./bin/JsonSource db.sqlite graph | ./bin/FwdConsistency | ./bin/JsonSink db.sqlite fwd  
./bin/JsonSource db.sqlite fwd | ./bin/TestGecodeBridge | ./bin/JsonSink db.sqlite csp
```

Pipeline de Evaluacion Rapida

```
./bin/SyntaxChecker input.json && \  
./bin/JsonToGraph input.json | \  
./bin/CSPEval | \  
./bin/TestGecodeBridge
```


Proximo: Leer `04_integracion_gecode.md` para entender el puente Pascal/C++/Gecode.